

Voxeria

Modding-System

Knoten-Editor, World Terminal und die Brücke dazwischen
Vollständige technische Referenz

Zweck. Dieses Dokument beschreibt das Modding-System des 2D-Voxel-Spiels Voxeria vollständig genug, dass ein Leser ohne Zugriff auf den Quelltext korrekte Mods entwerfen und das System technisch beurteilen kann. Es ist als Übergabe an eine andere KI gedacht und daher bewusst präzise statt werbend.

Stand. 63 Kartentypen, 27 Wert-Quellen, World Terminal v1.9. Alle Tabellen sind aus dem laufenden Spiel ausgelesen, nicht abgeschrieben.

Quelldateien. `voxeria-modding.js` (Katalog, Laufzeit, Editor), `voxeria-terminal.js` (Lexer, Parser, Compiler, Decompiler).

1. Die Kernidee: eine Sprache, zwei Oberflächen

Voxeria hat **nicht** zwei Modding-Systeme. Es hat eine einzige Sprache, die auf zwei Arten bedient wird:

- **Der Mod-Editor (Blueprint-Modus).** Karten werden aus einer Palette auf ein Board gezogen und mit Linien verbunden.
- **Das World Terminal.** Dieselben Karten, als Text getippt.

Beide erzeugen exakt dieselbe Datenstruktur und laufen durch exakt denselben Interpreter. Das ist keine Absichtserklärung, sondern strukturell erzwungen: es gibt im Terminal keinen Codepfad, der das Spiel erreicht, ohne vorher eine Katalogkarte geworden zu sein.

```
// Beide Wege enden an derselben Stelle
Palette → Karten + Linien ↘
                               ↳ {nodes, wires} → graphWalk() → Spielwelt
Skripttext → Lexer → Parser ↙      (ein Graph)      (ein Interpreter)
```

Die zentrale Tabelle heisst `NODE_CATALOG`. Sie definiert jede Karte mit Namen, Art und Parametern. Aus ihr werden erzeugt: die Editor-Palette, die Parameterfelder auf jeder Karte, die Terminal-Befehle, die Autovervollständigung, die Hilfetexte und die Validierung beim Laden fremder Mods. Eine neue Karte im Katalog erscheint automatisch in allen genannten Stellen.

63

Kartentypen

19

Ereignisse

10

Bedingungen

30

Aktionen

4

Schleifen

1.1 Warum es beide Oberflächen gibt

Das Board zeigt, *welches Vokabular existiert*: man sieht alle Karten, ohne etwas auswendig zu wissen. Das Terminal ist schneller, sobald man das Vokabular kennt. Zwölf Karten von Hand zu verdrahten ist mühsam, wenn man denselben Gedanken in drei Zeilen tippen kann.

2. Datenmodell

Ein Mod ist ein **Graph**: eine Menge Knoten und eine Menge gerichteter Linien.

```
{
  name: "Mein Mod",
  nodes: [ { id, type, x, y, params } , ... ],
  wires: [ { from, fromPort, to } , ... ]
}
```

2.1 Die vier Kartenarten

Art	Ausgänge	Bedeutung
event	out	Beginnt eine Kette. Nimmt keinen Eingang an.
cond	yes , no	Gabelt die Kette in zwei Zweige.
action	out	Tut etwas und gibt weiter.
loop	body , done	<code>body</code> läuft pro Durchgang, <code>done</code> einmal danach.

2.2 Regeln für Linien

- Pro Ausgang höchstens **eine** Linie. Ein gegabelter Ausgang hätte keine eindeutige Ausführungsreihenfolge.
- Ereignisse nehmen keine Eingangslinie an.
- Ein Graph darf mehrere unabhängige Ketten enthalten. Er ist trotzdem ein Mod.
- Beim Laden werden Linien auf unbekannte Knoten, Selbstverweise und doppelt belegte Ausgänge stillschweigend verworfen.

3. Wert-Slots: das wichtigste Konzept

Ein Parameter, der eine Zahl erwartet, verlangt nicht einfach eine Zahl. Er ist ein **Wert-Slot** und hat immer dieselbe gespeicherte Form:

```
{ s: Quelle, n: Zahl, v: Name }
```

Die Quelle entscheidet, was gelesen wird. Weil jeder Slot im gesamten Katalog jede Quelle akzeptiert, erweitert eine neue Quelle sofort jede Bedingung, jede Schleife und jede Aktion gleichzeitig. Das ist der Grund, warum das System mit 63 Karten so viel ausdrücken kann.

3.1 Alle 27 Quellen

Quelle	Bedeutung
a fixed amount	Eine feste Zahl, benutzt <code>n</code> .
a saved number	Eine gespeicherte Zahl (Variable), benutzt <code>v</code> als Name.
an item in a list	Eintrag einer Liste: <code>v</code> ist die Liste, <code>n</code> die Position (1-basiert).
an item in a list, at a position I calculate	Wie oben, aber die Position ist selbst ein verschachtelter Wert-Slot in <code>idx</code> . Genau eine Ebene tief, siehe 4.2.
how many in a list	Länge einer Liste, benutzt <code>v</code> .
seconds left on a countdown	Restsekunden des in <code>v</code> benannten Countdowns, 0 wenn keiner läuft.
Live-Messwerte aus dem laufenden Spiel (identisch mit <code>GRAPH_STATS</code>)	
health	Aktuelle Herzen des Spielers.
max health	Maximale Herzen.
depth	Tiefe unter der Oberfläche.
position x	Waagerechte Position.
time of day	Tageszeit als Zahl.
held count	Anzahl im gehaltenen Stapel.
blocks carried	Gesamtzahl getragener Blöcke.
creatures nearby	Kreaturen in der Nähe.
jumps left	Verbleibende Luftsprünge.
random 1-100	Zufallszahl, bei jedem Lesen neu gewürfelt.
players here	Andere Spieler in der Welt (Mehrspieler).
nearest player distance	Entfernung zum nächsten Spieler.
nearest player x	Dessen waagerechte Position.
nearest player y	Dessen senkrechte Position.
nearest player team	Dessen Team-Nummer.

Quelle	Bedeutung
<code>my team</code>	Eigene Team-Nummer.
<code>event player tag</code>	Der auslösende Spieler als Zahl (siehe 5.3). 0 ausserhalb eines Spieler-Ereignisses.
<code>event player team</code>	Dessen Mannschaft.
<code>event player x</code>	Dessen waagerechte Position in Feldern.
<code>event player y</code>	Dessen senkrechte Position in Feldern.
<code>event player distance</code>	Dessen Entfernung zu mir, in Feldern.

Warum die sechs Mehrspieler-Werte wichtig sind. Ohne sie kann ein Mod „Punkte zählen“ ausdrücken, aber niemals „Punkte dafür zählen, dass jemand anderes da ist“. Sie sind der Unterschied zwischen einer Regel und einem Minispiel.

Manche Slots kennen zusätzlich die Quelle `nothing`. Das bedeutet „dieser Wert wurde nicht angegeben“ und ist von der Zahl 0 unterscheidbar. Genutzt wird das von `Show-Text` (Text ohne Zahl), `Invoke-Signal` (nicht belegte Parameter) und `Send-Announcement`.

4. Zahlen und Listen

4.1 Gespeicherte Zahlen

- Namen werden auf Grossbuchstaben, Ziffern und Unterstrich normalisiert und auf 16 Zeichen gekürzt. `score`, `Score` und `SCORE` sind dieselbe Zahl.
- Jeder Wert wird auf **-9999 bis 9999** begrenzt und auf eine Nachkommastelle gerundet.
- Eine nie gesetzte Zahl liest sich als **0**, nicht als Fehler.
- Division durch null lässt die linke Seite unverändert, statt die Zahl auf 0 oder unendlich zu setzen.

4.2 Listen

Listen sind ein eigener Speicher neben den Zahlen. Eine Liste und eine Zahl mit demselben Namen sind zwei verschiedene Dinge.

- Höchstens **20 Einträge** pro Liste.
- Positionen sind **1-basiert**, weil Menschen sie über ein Feld „an position“ eingeben.
- Lesen ausserhalb der Liste ergibt **0**, kein Fehler.
- Eine Liste entsteht beim ersten Zugriff. Nichts muss deklariert werden.
- `set the item at` jenseits des Endes füllt mit Nullen auf. So lässt sich eine Liste als feste Slot-Menge nutzen, etwa vier Teampunkte.

Operationen von `Set-List`: `add to the end` · `set the item at` · `remove the item at` · `remove the last` · `clear it`.

4.3 Berechnete Positionen

Eine Position darf selbst gerechnet sein. Die Quelle `an item in a list, at a position I calculate` trägt dafür einen zweiten, verschachtelten Wert-Slot im Feld `idx`:

```
$LISTE[3]      // feste Position
$LISTE[$I]     // Position aus einer gespeicherten Zahl
$LISTE[@depth] // Position aus einem Live-Messwert
$LISTE[count]  // wie viele Eintraege
```

Genau eine Ebene, und das ist erzwungen. Der innere Slot wird gegen eine Quellenliste bereinigt, aus der diese Quelle selbst herausgefiltert ist. Ein Mod-Code, der eine zweite Verschachtelung behauptet, wird beim Laden automatisch auf eine feste Zahl zurückgestutzt. Die Rekursion kann also nicht tiefer gehen, auch nicht durch einen von Hand geänderten Code.

Was die Position **nicht** sein darf, ist ein ganzer Ausdruck wie `$I+1` : dafür müsste der Klammerinhalt richtig zerlegt werden statt als Text gelesen. Wer das braucht, rechnet vorher in eine Zahl.

5. Funktionen und Ankündigungen

Es gibt zwei Wege, Logik wiederzuverwenden. Sie sehen ähnlich aus und sind grundverschieden. Der Unterschied ist **nicht**, wer erreicht wird, sondern **wann**.

	Call by name (Funktion)	Announce (Ankündigung)
Zeitpunkt	sofort, verschachtelt im Aufrufer	im nächsten Frame, Aufrufer ist fertig
Rückgabe	ja, über <code>Return-Value</code>	nein
Parameter	bis zu 3	1 optionaler Wert
Grenze	8 Verschachtelungsebenen	flache Warteschlange, 32 Einträge
Empfänger	In beiden Fällen alle passenden Karten, auch in anderen Mods.	
Wofür	eine Frage mit Antwort	entkoppelte Teile, Kettenreaktionen

Warum das der praktisch wichtige Unterschied ist. Eine Kettenreaktion (Welle meldet sich, ein Zuhörer startet die nächste Welle) ist mit einem Funktionsaufruf nach 8 Schritten am Ende, weil jeder Aufruf auf dem Stapel des vorherigen sitzt. Als Ankündigung ist sie eine flache Warteschlange: jede Runde kostet genau einen Frame und läuft beliebig lange weiter. Eine Regel, die auf ihre eigene Ankündigung reagiert, kann sich ausserdem nicht selbst in denselben Frame zurückrufen, weil die Warteschlange vor der Zustellung komplett ausgetauscht wird.

5.1 Beispiel: dieselbe Aufgabe auf beide Arten

Als Funktion

```
block Schaden KRAFT WINKEL ART {
  $LETZTER = $KRAFT + $WINKEL
  Set-List LOG "add to the end" $ART
}

on Player jumps {
  Schaden 5 7 9
}
```

Als Ankündigung

```
on Player jumps {
  Send-Announcement SPRUNG 7
}

# voellig unabhaengiger Teil,
# darf in einem anderen Mod stehen
on Announced SPRUNG INFO {
  $GEZAEHLT += 1
}
```

5.2 Mehrspieler: Ereignisse statt Nachfragen

Andere Spieler waren lange nur *ablesbar*. Eine Regel musste selbst nachsehen und den Unterschied zum letzten Mal erraten. Vier Karten melden sich jetzt von sich aus:

Karte	Feuert
When another player arrives	wenn jemand in dieser Welt auftaucht
When another player leaves	Abmelden, Seed-Wechsel, Bann und Absturz
When a player enters a zone	beim Übergang in eine benannte Zone, nicht beim Drinstehen
When a player touches me	bei Nähe, höchstens alle 1,5 s je Spieler

Warum nicht an Firestore gehängt. Der naheliegende Weg wäre, die Änderungstypen `added` und `removed` zu benutzen. Das wäre in vier Fällen falsch: ein Spieler auf einem fremden Seed meldet ebenfalls `added`; wer den Seed wechselt, meldet `modified` statt Beitritt oder Weggang; ein Gebannter wird gesondert gefiltert; und **wer abstürzt, meldet gar nichts**, sein Dokument bleibt liegen und wird nur still.

Die Engine hat all das bereits richtig gelöst, nämlich in `otherPlayers`. Deshalb wird diese Liste zwischen zwei Frames verglichen, statt Firestore zu belauschen. Ein Abstürzter fällt nach derselben 8-Sekunden-Frist heraus, die die Spielerzählung ohnehin benutzt.

5.3 Wie ein Spieler in einen Wert-Slot passt

Der Katalog kennt ausschliesslich Zahlen, eine Firebase-Kennung ist aber eine Zeichenkette. Sie direkt einzusetzen würde den Datentyp des ganzen Systems aufbrechen. Stattdessen wird die Kennung deterministisch auf eine Zahl von 1 bis 9999 gestreut (FNV-1a):

```
graphPlayerTag(uid) // gleiche Kennung, gleiche Zahl, auf JEDEM Client
```

Kein Server, keine Absprache, keine Verteilung. Damit lassen sich Spieler vergleichen und als Listeneintrag merken, ohne dass je Text durch den Katalog wandert. Gelesen wird das über die fünf `event player`-Quellen, die ausserhalb eines Spieler-Ereignisses 0 liefern.

5.4 Countdowns und Punktetafel

Beide sind bewusst so gebaut, dass sie **keinen zusätzlichen Abgleich** brauchen.

- **Ein Countdown ist rein örtlich.** Gespeichert wird ein Endzeitpunkt, heruntergezählt wird gegen die eigene Uhr. Starten alle Clients denselben Countdown im selben Moment, läuft er synchron, weil jeder dieselbe verstreichende Zeit misst. Das ist dasselbe Muster, das die Arena mit `remainingS()` schon benutzt, und es funktioniert dadurch in jeder Welt, auch allein. Ehrlich benannt: wird er durch ein Ereignis gestartet, das nicht alle gleichzeitig sehen, laufen auch die Uhren auseinander. Wer echten Gleichlauf braucht, startet ihn aus `When the match starts`, das der Host treibt.
- **Die Punktetafel zeigt nur, was ohnehin abgeglichen wird.** Punkte vergibt `Score points`, die Arena verteilt sie bereits über je ein eigenes Dokument pro Spieler. Die Karte ist reine Anzeige, Team-Summen entstehen lokal aus denselben Einträgen. Es gibt deshalb keinen zweiten Weg, auf dem etwas auseinanderlaufen könnte.

6. Der vollständige Katalog

Alle 63 Karten mit ihrem Editor-Namen, ihrem Terminal-Befehl und ihren Parametern. Die Spalte „Terminal“ nennt bei zusammengelegten Karten beide Verben: `Give-Item` und `Take-Item` sind dieselbe Karte, deren Auswahlfeld der Befehlsname bereits beantwortet.

Ereignisse 19

Beginnen eine Kette. Die drei Karten mit „Before“ laufen, während das Spiel noch entscheidet, und können den Ausgang ändern.

Karte im Editor	Terminal	Parameter
When the world starts onWorldStart	WorldStart	keine
Every N seconds onTimer	Timer	• seconds ZAHL 1 bis 60

Karte im Editor	Terminal	Parameter
When the player ... a block onBlock	Block	- how „the player“ AUSWAHL <i>touches · mines · places</i> - block „the block“ BLOCK
When the player ... onPlayer	Player	how „the player“ AUSWAHL <i>jumps · gets hurt · dies</i>
When entering a dimension onEnterDim	EnterDimension	dim „dimension“ AUSWAHL <i>OVERWORLD · GOLD · LAVA · OCEAN · VOID · ERG</i>
When the time of day changes onDayPhase	DayPhase	phase „when“ AUSWAHL <i>night falls · day breaks</i>
When called by name onSignal	Signal	- signal NAME - argVar „receiving“ NAME - argVar2 „and“ NAME - argVar3 „and“ NAME
When something is announced onAnnounced	Announced	- event „announcement“ NAME - valueVar „value into“ NAME
When another player arrives onPlayerJoin	PlayerJoin	keine
When another player leaves onPlayerLeave	PlayerLeave	keine
When a player enters a zone onPlayerEnterZone	PlayerEnterZone	zone NAME
When a player touches me onPlayerTouch	PlayerTouch	range „within“ ZAHL 1 bis 8
When a countdown ends onCountdownEnd	CountdownEnd	timer „countdown“ NAME
When a creature ... onCreature	Creature	how „a creature“ AUSWAHL <i>dies · is hurt · attacks</i>
Before the player is hurt onBeforeHurt	BeforeHurt	keine
Before a block breaks onBeforeMine	BeforeMine	block BLOCK
When the player picks up onPickup	Pickup	block BLOCK
When the match starts onMatchStart	MatchStart	keine

Karte im Editor	Terminal	Parameter
When the match ends onMatchEnd	MatchEnd	keine

Bedingungen 10

Wählen den Ausgang `yes` oder `no`. Alles, was einen Fehler wirft oder etwas Unbekanntes nennt, fällt auf `no`.

Karte im Editor	Terminal	Parameter
Random chance ifChance	Chance	<code>percent</code> „chance %“ ZAHL 1 bis 99
If a number ... ifCompare	Compare	<code>a</code> „if“ WERT-SLOT -9999 bis 9999, 1 Nachkommastelle <code>op</code> AUSWAHL <i>is exactly · is not · is more than · is less than · is at least · is at most</i> <code>b</code> WERT-SLOT -9999 bis 9999, 1 Nachkommastelle
If the player ... a block ifBlock	HasBlock	<code>how</code> „the player“ AUSWAHL <i>is holding · is standing on · is carrying at least · is the block involved</i> <code>block</code> „the block“ BLOCK <code>count</code> „at least“ WERT-SLOT 1 bis 999
If right now ... ifState	State	<code>state</code> „if“ AUSWAHL <i>it is night · the player is in water · the player is on the ground</i>
If the world's ... is ifWorldIs	World	<code>what</code> „the“ AUSWAHL <i>dimension · biome · weather</i> <code>value</code> „is“ AUSWAHL <i>(haengt vom Nachbar-Parameter ab)</i>
If the block nearby is ifBlockAt	BlockAt	<code>dx</code> ZAHL -12 bis 12 <code>dy</code> ZAHL -12 bis 12 <code>block</code> BLOCK
If wearing armor of ifWearingArmor	WearingArmor	<code>dim</code> AUSWAHL <i>GOLD · LAVA · OCEAN · VOID</i>
If inside the arena ifInArena	InArena	keine
If my score is at least ifScoreAtLeast	ScoreAtLeast	<code>points</code> WERT-SLOT 1 bis 999
If I am in the lead ifLeading	Leading	keine

Schleifen 4

Haben zwei Ausgänge. `wait` innerhalb eines Schleifenrumpfs ist wirkungslos und meldet das einmal in der Konsole.

Karte im Editor	Terminal	Parameter
Repeat repeatTimes	repeat	<code>count</code> „how many times“ WERT-SLOT 1 bis 200
Repeat while repeatWhile	while	<code>a</code> „while“ WERT-SLOT -9999 bis 9999, 1 Nachkommastelle <code>op</code> AUSWAHL <i>is exactly · is not · is more than · is less than · is at least · is at most</i> <code>b</code> WERT-SLOT -9999 bis 9999, 1 Nachkommastelle

Karte im Editor	Terminal	Parameter
For each item I'm carrying forEachItem	foreach	• itemVar „item into“ NAME • countVar „count into“ NAME
For each item in a list forEachInList	forlist	• list NAME • itemVar „value into“ NAME • indexVar „position into“ NAME

Aktionen 30

Tun etwas und geben weiter.

Karte im Editor	Terminal	Parameter
Give or take items changeItems	Give-Item / Take-Item	- how AUSWAHL Give · Take away - block „item“ BLOCK - count „how many“ WERT-SLOT 1 bis 64
Give or take the block involved changeInvolvedItem	Give-Involved / Take-Involved	- how AUSWAHL Give · Take away - count „how many“ WERT-SLOT 1 bis 64
Show text or a number showText	Show-Text	- where „show“ AUSWAHL as a banner · floating on the player - text TEXT - number „and“ WERT-SLOT -9999 bis 9999, 1 Nachkommastelle darf leer bleiben - color „colour“ AUSWAHL white · red · orange · gold · green · cyan · blue · purple · pink
Heal or hurt the player changeHealth	Heal-Player / Hurt-Player	- how AUSWAHL Heal · Hurt - amount „by“ WERT-SLOT 1 bis 12
Set a player stat setStat	Set-Stat	- stat „set“ AUSWAHL move speed · jump power · gravity · max hearts · reach bonus · air jumps · mining yield · damage taken · 3x3 mining · hazard immunity - to WERT-SLOT 0 bis 15, 1 Nachkommastelle
Move the player movePlayer	Teleport-Player / Launch-Player	- how AUSWAHL teleport by · launch with force - dx „sideways“ WERT-SLOT -40 bis 40 - dy „up / down“ WERT-SLOT -40 bis 40
Set the world's ... setWorld	Set-World	- what „set the“ AUSWAHL time of day · weather - value „to“ AUSWAHL (haengt vom Nachbar-Parameter ab)
Spawn my creature spawnCreature	Spawn-Creature	creature KREATUR
Set up my creature setCreatureBehavior	Set-Creature	- creature KREATUR - move „moves by“ AUSWAHL patrol · jump · fly - size WERT-SLOT 8 bis 36 - speed WERT-SLOT 0.2 bis 1.6, 1 Nachkommastelle - rarity „how common“ WERT-SLOT 1 bis 8 - biome „lives in“ AUSWAHL any · FOREST · SNOW - traits AUSWAHL none · glows · trail · both
Set up my creature's fight setCreatureCombat	Set-Combat	- creature KREATUR - health WERT-SLOT 0 bis 40 - damage „hits for“ WERT-SLOT 1 bis 12 - attack „attacks by“ AUSWAHL none · touch · charge · leap · shoot - aggro „notices within“ WERT-SLOT 0 bis 24

Karte im Editor	Terminal	Parameter
Emit particles emitParticles	Emit-Particles	at AUSWAHL <i>the player · the block involved</i> color „colour“ AUSWAHL <i>white · red · orange · gold · green · cyan · blue · purple · pink</i> amount „how many“ WERT-SLOT 1 bis 60 power „spread“ WERT-SLOT 1 bis 14
Shake the screen shake	Invoke-Shake	power WERT-SLOT 2 bis 30
Play a sound playSound	Play-Sound	sound AUSWAHL <i>reward · hurt · place · explode · portal · jump · click · alarm · denied · buy</i>
Set or change a number changeVar	Set-Number	name „number“ NAME op AUSWAHL <i>set to · add · subtract · multiply by · divide by · smallest of · largest of</i> to WERT-SLOT -9999 bis 9999, 1 Nachkommastelle
Change a list changeList	Set-List	list NAME how AUSWAHL <i>add to the end · set the item at · remove the item at · remove the last · clear it</i> value WERT-SLOT -9999 bis 9999, 1 Nachkommastelle position „at position“ WERT-SLOT 1 bis 20
Wait, then carry on wait	Wait-Seconds	seconds „wait“ WERT-SLOT 0.1 bis 60, 1 Nachkommastelle
Stop it from happening preventIt	Stop-Event	keine
Change the amount to setEventAmount	Set-Amount	amount „to“ WERT-SLOT 0 bis 24
Make this block drop remapDrop	Set-Drop	from BLOCK to BLOCK
Set how a block mines setBlockMining	Set-Mining	block BLOCK hardness WERT-SLOT 1 bis 8 sound AUSWAHL <i>stone · denseStone · ore · shinyOre · crystal · wood · glass · dirtEarth</i>
Fill a box with fillArea	Fill-Area	block „material“ BLOCK w „width“ WERT-SLOT 1 bis 9 h „height“ WERT-SLOT 1 bis 9
Call by name callSignal	Invoke-Signal	signal NAME arg „passing“ WERT-SLOT -9999 bis 9999, 1 Nachkommastelle darf leer bleiben arg2 „and“ WERT-SLOT -9999 bis 9999, 1 Nachkommastelle darf leer bleiben arg3 „and“ WERT-SLOT -9999 bis 9999, 1 Nachkommastelle darf leer bleiben result „result into“ NAME

Karte im Editor	Terminal	Parameter
Mark a zone here markZone	Set-Zone	- zone NAME - w „width“ WERT-SLOT 1 bis 64 - h „height“ WERT-SLOT 1 bis 64
Start or stop a countdown setCountdown	Start-Countdown / Stop-Countdown	- how AUSWAHL <i>start · stop</i> - timer „countdown“ NAME - seconds „for“ WERT-SLOT 1 bis 600
Show the scoreboard showBoard	Show-Board	- how AUSWAHL <i>per player · per team · hide it</i> - title „titled“ TEXT
Announce something happened announce	Send-Announcement	- event „announcement“ NAME - value „with“ WERT-SLOT -9999 bis 9999, 1 Nachkommastelle darf leer bleiben
Return a value returnValue	Return-Value	value „return“ WERT-SLOT -9999 bis 9999, 1 Nachkommastelle
Score points addScore	Add-Score	points WERT-SLOT -99 bis 99
End the round now endRound	Stop-Round	keine
Join a team setTeam	Join-Team	team WERT-SLOT 0 bis 8

7. Das World Terminal

Erreichbar mit **Strg+T** oder über die Schaltfläche `>_ Terminal` im Mod-Editor. **F9** schaltet auf eine schlichte Darstellung um.

7.1 Grammatik

```
script      := stmt*
stmt        := onBlock | blockDef | ifStmt | loopStmt | assign | call
onBlock     := 'on' NAME args '{' stmt* '}'
blockDef    := 'block' NAME NAME{0,3} '{' stmt* '}'
ifStmt      := 'if' bool '{' stmt* '}' ('else' ifStmt | 'else' '{' stmt* '}')?
loopStmt    := 'repeat' expr '{' stmt* '}'
              | 'while'  cmp  '{' stmt* '}'
              | 'foreach' args '{' stmt* '}'
              | 'forlist' args '{' stmt* '}'
assign      := NAME ('=' | '+=' | '-=' | '*=' | '/=') expr
call        := NAME args
args        := (value | '(' expr ')') | '-' NAME (value | '(' expr ')')*

bool        := bool 'or' bool | bool 'and' bool | 'not' bool
              | '(' bool ')' | condCall | cmp
cmp          := expr ('==' | '!=' | '<' | '>' | '<=' | '>=') expr
expr         := expr ('+' | '-') expr | expr ('*' | '/') expr | '-' expr
              | NUMBER | '$NAME' | '@messwert' | listRead | '(' expr ')'
listRead    := '$NAME' '[' (NUMBER | 'count' | 'left' | '$NAME' | '@messwert') ']'
```

7.2 Sigils

Schreibweise	Bedeutung
<code>\$NAME</code>	Eine gespeicherte Zahl.
<code>@messwert</code>	Ein Live-Messwert, etwa <code>@health</code> oder <code>@depth</code> .
<code>\$NAME[3]</code>	Der dritte Eintrag der Liste <code>NAME</code> .
<code>\$NAME[\$I]</code>	Der Eintrag an der Position, die <code>\$I</code> gerade sagt.
<code>\$NAME[@depth]</code>	Dasselbe mit einem Live-Messwert als Position.
<code>\$NAME[count]</code>	Wie viele Einträge die Liste hat. <code>[length]</code> ist gleichbedeutend.
<code>\$NAME[left]</code>	Restsekunden des Countdowns <code>NAME</code> . Dritter Namensraum in derselben Klammer, das Schlüsselwort sagt, worum es geht.
<code>-Parameter</code>	Benannter Parameter statt Position, etwa <code>-How Give</code> .
<code>#</code>	Kommentar bis Zeilenende.

Die Sigils existieren, damit die Quelle eines Werts im Text sichtbar bleibt: `Give-Item gold 3` und `Give-Item gold $SCORE` unterscheiden sich auf einen Blick, was im Editor das Auswahlfeld leistet.

7.3 Zuweisungen

`$SCORE = 5` und `Set-Number SCORE "set to" 5` erzeugen dieselbe Karte. Ausdrücke werden echt übersetzt: `$A + $B * 2` wird in eine Folge von `Set-Number`-Karten mit Zwischenwerten zerlegt, genau wie ein Compiler Arithmetik in Register

auflöst. Es gibt im Interpreter keinen Ausdrucksauswerter und es soll auch keinen geben.

`and` und `or` werden reine Verdrahtung: in `A and B` hängt Bs Karte am `yes`-Ausgang von A. Damit ist die Kurzauswertung kostenlos, ohne dass die Laufzeit sie kennen muss.

7.4 Eingebaute Befehle

Dies sind die einzigen Befehle, die **keine** Katalogkarten sind. Jeder betrifft das Terminal selbst, keiner verändert die Welt.

Befehl	Wirkung
<code>help [befehl]</code>	Alle Befehle, oder einer im Detail.
<code>blocks [filter]</code>	Blocknamen, die ein Parameter annehmen kann.
<code>stats</code>	Jeder Live-Messwert mit seinem aktuellen Wert.
<code>vars</code>	Jede gespeicherte Zahl, die die Mods gerade halten.
<code>rules</code>	Die Regeln, die diese Sitzung installiert hat.
<code>reset</code>	Entfernt alle Regeln dieser Sitzung.
<code>mod list</code>	Alle gespeicherten Mods.
<code>mod save <name></code>	Sitzung dauerhaft als Mod speichern.
<code>mod open</code>	Sitzung auf dem Editor-Board anzeigen.
<code>mod import <name></code>	Gespeicherten Mod als Skripttext ausgeben.
<code>mod code</code>	Teilbaren VXG1-Code der Sitzung.
<code>logic [show delete]</code>	Eigene Bausteine verwalten.
<code>history</code> / <code>clear</code> / <code>exit</code>	Verlauf, Bildschirm leeren, schliessen.

7.5 Einfärbung nach Typ

Die Skriptausgabe ist nach der **Art** eines Befehls eingefärbt, nicht nach dem einzelnen Befehl: alle Ereignisse teilen sich eine Farbe, alle Bedingungen eine, und so weiter. Die Zuordnung kommt aus `NODE_CATALOG`, also aus derselben Tabelle wie die Palette. Eine neue Karte bekommt ihre Farbe damit von selbst und kann nicht vergessen werden.

Art	Farbe	Beispiele
Ereignis	Violett	<code>PlayerTouch</code> , <code>WorldStart</code>
Bedingung	Bernstein	<code>Chance</code> , <code>Compare</code>
Aktion	Blau	<code>Give-Item</code> , <code>Add-Score</code>
Schleife	Grün	<code>repeat</code> , <code>forlist</code>
Eingebauter Befehl	Rosa	<code>help</code> , <code>mod</code>
Eigener Baustein	Akzent	selbst definierte

Die vier Kartenarten benutzen bewusst dieselben Farbtöne wie die Karten im Editor, nur aufgehellt, weil hier Text auf dunklem Grund steht statt einer gefüllten Kopfzeile. **Was auf dem Board grün ist, ist im Terminal grün.** Dazu kommen eigene Töne für `$Zahlen` , `@Messwerte` , Zeichenketten, Zahlen und Kommentare.

Im schlichten Modus (F9) bleiben die Farben erhalten, stark entsättigt: welcher Art ein Befehl ist, ist Information und keine Verzierung. Das ist dieselbe Regel, nach der auch OK und ERR dort ihre Farbe behalten.

7.6 Eigene Bausteine

`block Name A B C { ... }` definiert einen benannten Baustein mit bis zu drei Werten. Darunter ist das keine neue Maschinerie, sondern genau die Kombination aus `when called by name` und `call by name`. Ein so definierter Baustein erscheint **sofort in der Editor-Palette** unter „My blocks“ und lässt sich dort wie jede andere Karte auf das Board ziehen.

8. Die Synchronisation zwischen Board und Terminal

Dies ist der Teil, der die beiden Oberflächen zu einem System macht. Wichtig vorweg, weil es leicht falsch erinnert wird: die Synchronisation ist **nicht in beide Richtungen gleich**. Sie ist in einer Richtung automatisch und in der anderen ein bewusster Befehl.

8.1 Terminal → Board (automatisch)

Geprüft wird an genau zwei Momenten: beim **Öffnen des Mod-Editors** und beim **Schliessen des Terminals** auf einen darunter offenen Editor. Es gilt eine oberste Regel: **niemals Arbeit zerstören, die auf dem Board liegt**.

Lage	Verhalten
Board ist leer	Die Regeln der Sitzung werden direkt geladen, mit Hinweis „Showing your Terminal rules“. Es kann nichts verloren gehen.
Board enthält bereits etwas	Ein violettes Band erscheint: „N rules from the Terminal aren't on this board yet“ mit der Schaltfläche Add to this board . Erst ein Klick fügt sie hinzu.
Alles ist schon da	Kein Band. Wer einmal hinzugefügt hat, wird nicht erneut gefragt.

Das Hinzufügen kopiert die Knoten **unterhalb** der vorhandenen, sodass sich nichts überlagert, und legt sie als **einen einzigen Undo-Schritt** ab. Die Linien werden nur untereinander neu verknüpft, also genauso getrennt, wie zwei in verschiedenen Zeilen getippte Ketten es ohnehin sind.

`mod open` ist die ausdrückliche Variante: sie lädt die Sitzung auf das Board, schliesst das Terminal und öffnet den Editor, falls er zu war.

8.2 Board → Terminal (auf Befehl)

Diese Richtung läuft nicht automatisch. Das Terminal spiegelt das Board nicht laufend. Wer eine Karte auf das Board zieht, sieht sie nicht von selbst als Text im Terminal erscheinen.

Der Weg zurück ist: den Mod speichern, dann `mod import <name>`. Der Decompiler gibt ihn als Skripttext aus, den man lesen, bearbeiten und wieder einfügen kann. `logic show` tut dasselbe für einen eigenen Baustein.

8.3 Der Decompiler

Der Rückweg ist die ehrliche Probe auf die Brücke: ein durch Ziehen gebauter Graph muss als Skript zurückkommen, das ein Mensch hätte tippen können. Er gibt bewusst lesbare Formen aus, nicht die interne Schreibweise:

Auf dem Board	Als Text
Wert-Slot „a saved number“ SCORE	<code>\$SCORE</code>
Wert-Slot „an item in a list“	<code>\$LISTE[3]</code>
Wert-Slot „how many in a list“	<code>\$LISTE[count]</code>
Karte „Set or change a number“, Operation „add“	<code>\$SCORE += ...</code>
„When called by name“ unter einem selbst vergebenen Namen	<code>block Name ... { }</code>

Wo eine Board-Form keine Schreibweise hat, etwa zwei Ketten, die in der Mitte zusammenlaufen, schreibt der Decompiler einen Kommentar statt etwas auszugeben, das sich nicht zurückübersetzen liesse.

8.4 Was in beide Richtungen sofort gilt

- Ein mit `block` definierter Baustein steht **sofort** in der Editor-Palette unter „My blocks“.
- Eine Karte, die aus dem Terminal stammt, ist auf dem Board von einer gezogenen nicht zu unterscheiden. Es gibt keine Herkunftsmarkierung, weil es keinen Unterschied gibt.
- Beide Wege speichern in dieselbe Mod-Bibliothek und erzeugen denselben teilbaren Code.

8.5 Der Probelauf im Editor

Das Board hat einen Trockenlauf, der eine Kette Schritt für Schritt beschreibt, statt sie auszuführen. Grund: die Aktionen greifen in die echte Welt, und ein „Loch sprengen“ während des Bauens wäre ein echtes Loch an einer Stelle, die niemand gewählt hat. Schleifen werden als **ein** Durchgang gezeigt, damit ein `Repeat 200` nicht zweihundert gleiche Zeilen erzeugt. Bedingungen melden, ob ihr Ausgang gewürfelt oder angenommen wurde. Nicht verdrahtete Ausgänge werden ausdrücklich genannt, denn ein leerer `no`-Ausgang ist der häufigste Grund dafür, dass ein Mod „manchmal nichts tut“.

9. Laufzeit und Grenzen

9.1 Wie eine Kette läuft

Ein Ereignis startet `graphWalk`. Die Funktion folgt den Linien, führt Aktionen aus, wählt bei Bedingungen einen Ausgang und ruft sich für Schleifenrumpfe selbst auf. Alle diese Aufrufe teilen sich **ein einziges** Schrittbudget. Zehn ineinander verschachtelte Schleifen bekommen also nicht jeweils ein eigenes Budget, sondern zusammen dasselbe. Das ist die Eigenschaft, die Verschachtelung überhaupt sicher macht.

9.2 Alle harten Grenzen

Grenze	Wert	Warum
<code>GRAPH_MAX_NODES</code>	150	Karten pro Mod.
<code>GRAPH_MAX_WIRES</code>	225	Linien pro Mod.
<code>GRAPH_MAX_STEPS</code>	2000	Schritte je Auslösung, über alle Verschachtelungen hinweg.
<code>GRAPH_MAX_LOOP_ITERS</code>	200	Durchgänge je Schleife. Gilt auch für <code>while</code> , dessen Bedingung nie falsch werden könnte.
<code>GRAPH_MAX_SIGNAL_DEPTH</code>	8	Verschachtelte Funktionsaufrufe. Verhindert unendliche Rekursion.
<code>GRAPH_MAX_PENDING</code>	64	Gleichzeitig wartende <code>wait</code> -Ketten.
<code>GRAPH_MAX_HOOK_DEPTH</code>	4	Tiefe, wenn eine Regel das Ereignis auslöst, an dem sie selbst hängt.
<code>GRAPH_MAX_LIST</code>	20	Einträge je Liste.
<code>GRAPH_MAX_ANNOUNCE</code>	32	Wartende Ankündigungen.
<code>GRAPH_MAX_ZONES</code>	8	Benannte Zonen gleichzeitig.
<code>GRAPH_MAX_COUNTDOWNS</code>	8	Laufende Countdowns gleichzeitig.
<code>GRAPH_TOUCH_MAX_RANGE</code>	8	Weiteste Reichweite für „berührt mich“, in Feldern.

Warum die Kartengrenze bei 150 steht und nicht mehr bei 60. Gemessen wurde, was die alte Grenze eigentlich geschützt hat, und es war nicht die Ausführung: ein Graph mit 400 Karten läuft genauso schnell wie einer mit 60, weil das Schrittbudget die Arbeit deckelt und nicht die Kartenzahl. Der Scan über 1200 Karten kostet 0,0017 ms je Frame, ein Hundertstel Prozent des Frame-Budgets.

Die echten Kosten lagen beim Teilen und beim Zeichnen, und beide sind inzwischen kleiner als vorher: ein Mod mit 150 Karten ist komprimiert rund 4 KB gross (60 Karten waren unkomprimiert 10,6 KB) und öffnet sich im Editor in rund 47 ms (60 Karten brauchten 100 ms). `GRAPH_MAX_STEPS` bleibt bei 2000, denn es ist nachweislich das, was tatsächlich schützt.

Wird eine Grenze erreicht, ist das Ergebnis immer „dieser Schritt tut nichts“ und nie ein Absturz. Bei einem geteilten Mod von einer fremden Person ist das die richtige Reaktion.

9.3 Die drei Karten, die das Spiel überstimmen

Alle anderen Ereignisse melden etwas, das bereits passiert ist. Diese drei laufen, während das Spiel noch entscheidet: `Before the player is hurt`, `Before a block breaks`, `When the player picks up`. An ihnen hängende Ketten können mit `Stop-Event` den Vorgang abbrechen oder mit `Set-Amount` die Menge ändern.

Technisch geschieht das **ohne Eingriff in die Engine**: das Modding-System ersetzt zur Laufzeit die Funktionen `takeDamage`, `breakSingleBlock` und `addToInventory` durch eigene Umhüllungen. Weil alle Dateien sich einen globalen Namensraum teilen, lösen sämtliche bestehenden Aufrufstellen danach auf die Umhüllung auf.

9.4 Standing rules

Zwei Aktionen setzen keine einmalige Wirkung, sondern eine dauerhafte Regel: `Set-Drop` (was ein Block beim Abbauen ergibt) und `Set-Mining` (Härte und Klang). Sie schreiben in eine Überlagerungsschicht, nie in die Tabellen der Engine selbst, damit die Regel eines Mods nicht in eine Sitzung überläuft, die später ohne diesen Mod lädt.

10. Codeformate

Präfix	Inhalt	Aufbau
VXG2 -	Ein Knotengraph, komprimiert	Dasselbe JSON, aber durch einen LZSS-Kompressor und dann base64url. Spart rund 82 %. Das schreibt das Spiel heute.
VXG1 -	Ein Knotengraph, unkomprimiert	base64url eines JSON mit Knoten und Linien. Wird weiterhin gelesen und immer gelesen werden, geschrieben wird es nicht mehr.
VXM3 -	Ein klassischer Mod	base32 fester Bitfelder. Ein ganzer Mod passt in etwa 20 Zeichen.
VXL1 -	Ein Loadout	Ein VXM3-Mod, gefolgt von beliebig vielen Teile-Codes, jeweils vollständig eingebettet.

Jedes Teil reist **vollständig** mit, nicht als Verweis auf einen Server. Damit gilt die Zusage des Formats: ein eingefügter Code ist sofort spielbar, ohne Netzwerk, und kann nicht verrotten, wenn jemand sein veröffentlichtes Teil löscht. Der Preis ist Länge.

Der Kompressor ist bewusst selbst geschrieben und arbeitet **ohne Wörterbuch**. `CompressionStream` wäre asynchron, Kodieren und Dekodieren werden aber an einem Dutzend Stellen synchron aufgerufen. Und ein Wörterbuch aus dem Katalog müsste sich mit dem Katalog ändern, womit jeder vorher geteilte Code etwas anderes bedeuten würde. Die Rückverweise stehen im Datenstrom selbst, ein VXG2-Code kann deshalb nicht verrotten.

10.1 Rückwärtskompatibilität

- Ein Kartentyp, den es früher gab, wird beim Laden auf seinen Nachfolger umgeschrieben und behält seine Einstellung.
- Ein unbekannter Kartentyp wird verworfen, nicht geraten. Ein Mod aus einer neueren Version lädt dadurch als „die Teile, die diese Version versteht“, statt die Annahme ganz zu verweigern.
- Ein fehlender Parameter bekommt den Standardwert. Neu hinzugefügte Parameter sind deshalb so gewählt, dass ihr Standard „nicht angegeben“ bedeutet und ein alter Code exakt weiterhin das tut, was er immer tat.
- Wert-Quellen und Messwerte werden als **Zeichenkette** gespeichert, nicht als Index. Anhängen ist daher sicher, Umbenennen wäre es nicht.

11. Sicherheitsmodell

Ein Mod kann **niemals Code enthalten**. Er ist immer nur eine Auswahl aus dem geschlossenen Katalog. Ein eingefügter Code wird nicht ausgeführt, sondern in Katalogkarten zerlegt, und jeder Parameter wird auf das begrenzt, was seine Definition erlaubt. Es gibt kein `eval`, kein erreichbares Wirtsobjekt und keinen Netzwerkzugriff.

Ein syntaktisch perfektes Skript, das etwas benennt, das es im Spiel nicht gibt, ist ein Parserfehler, keine Lücke. Deshalb ist ein von Fremden geteilter Mod in dem Moment sicher, in dem er eingefügt wird.

12. Bekannte Grenzen

Ehrlich benannt, weil ein Beurteiler sie ohnehin sucht:

- **Nur Zahlen als Datentyp.** Listen halten Zahlen, keinen Text und keine verschachtelten Listen. Ein Spieler wird deshalb als gestreute Zahl dargestellt und nicht mit Namen (siehe 5.3).
- **Keine eigenen Oberflächenelemente.** Ein Mod kann Text als Banner oder schwebend zeigen und die Punktetafel einblenden, aber kein eigenes Menü bauen.
- **Eine Listenposition darf gerechnet, aber kein Ausdruck sein.** `$L[$I]` geht, `$L[$I+1]` nicht. Wer das braucht, rechnet vorher in eine Zahl.
- **Punktetafel nur in der Arena.** Ausserhalb eines Matches werden keine Punkte abgeglichen, die Karte ist dort still wirkungslos. In anderen Welten lässt sich eine Tafel aus Listen und Text selbst bauen.
- **„Berührt mich“ ist Nähe, kein Tastendruck.** Das Spiel hat keine Taste für das Berühren anderer Spieler; ein Eingriff dafür wäre Engine-Arbeit, keine Katalog-Arbeit.
- **150 Karten pro Mod.** Grössere Vorhaben verteilen sich auf mehrere Mods, was durch Ankündigungen tragfähig ist, aber eine Grenze bleibt.

Einordnung

Das System ist bewusst kein tModLoader und kein Datapack-System: dort schreibt man echten Code und kann alles. Es ist strukturell näher an Scratch oder einem visuellen Skriptsystem, mit Ereignissen, Bedingungen, Schleifen, Variablen, Listen samt berechneten Positionen, Funktionen mit bis zu drei Parametern, entkoppelten Ankündigungen, Mehrspieler-Ereignissen, Zonen, Countdowns und Zugriff auf den Spielzustand. Der Deckel ist absichtlich gesetzt, damit ein geteilter Mod ohne Prüfung sicher ausführbar bleibt.

Ein vollständiges Fangen-Minispiel mit Runde, Basis-Zone, Punkten und Endstand passt damit in ein knappes Skript:

```
on MatchStart {
  Start-Countdown ROUND 60
  Show-Board "per player" "Fangen"
  Set-Zone SAFE 10 10
}

on PlayerTouch 2 {
  Add-Score 1
  Play-Sound reward
}

on PlayerEnterZone SAFE {
  $GERETTET += 1
}

on CountdownEnd ROUND {
  Show-Board "per team" "Endstand"
  Stop-Round
}
```